



RAGHU ENGINEERING COLLEGE

Accredited by NAAC and NBA, Affiliated to JNTGV- Gurajada, Dakamarri,
Bheemunipatnam (Mandal), Visakhapatnam.

Railway track Defect Detection

Batch no : 05

Team Members:

L. Amrutha Yashaswini (22981A4286)
K. Sravani (22981A4280)
S.Charan (22981A42C9)
K.Lokesh (23985A4206)

Guide Name:

Mr. V. Avinash
(Assistant Professor)

Identifying a Suitable Dataset

We explored academic papers and came across: MDPI Paper (2024)

Used the Kaggle dataset: Railway Track Fault Detection Dataset

This dataset was used in research for:

- CNN-based defect detection
- Binary fault classification
- Performance benchmarking

So we adopted it as our base dataset.

- Dataset was manually collected from various railway tracks (e.g., Bangladesh locations) in related academic work.
- Includes images of faults in rails and fasteners used for binary defect detection.
- Researchers have augmented the dataset to increase defect variety and address class imbalance (e.g., using oversampling techniques like SMOTE).
- Some studies split data into subsets focusing on specific fault types (e.g., rail segment faults vs fastener faults).

Dataset Extension & Segmentation Curation

- Upon analyzing the Kaggle dataset, we observed that it provides only binary image-level labels, without pixel-level segmentation masks or multi-class defect differentiation.
- Since our objective is to implement panoptic segmentation, we require:
 - Pixel-level masks
 - Instance-level annotations
 - Multi-class defect labeling
- Therefore, we initiated the process of curating our own segmentation dataset.
- Prior to annotation, we evaluated multiple annotation platforms:
 - Supervisely
 - Roboflow
 - Label Studio
- The evaluation criteria included:
 - Polygon and mask annotation capabilities
 - Supported export formats
 - Team collaboration features
 - YOLO compatibility
 - Dataset version control and management
- Based on these considerations, we selected **Supervisely** as our primary annotation tool.

Segmentation Annotation Workflow

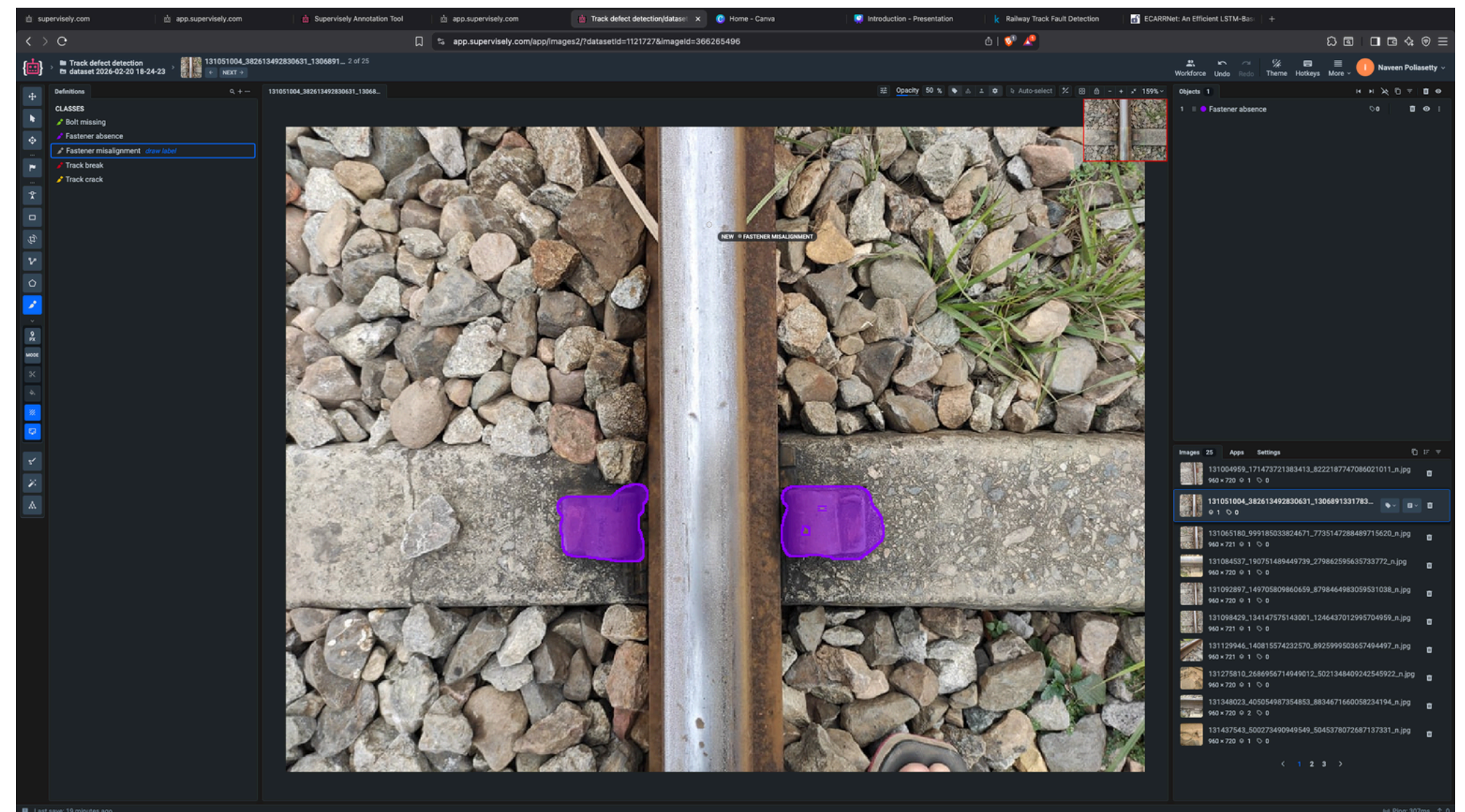
- We manually annotate defect regions using polygon-based masking.
- Each defect instance is outlined with high-precision pixel boundaries.
- Annotations are performed at instance level, not image level.

Class Definition

- Custom class taxonomy defined before annotation.
- Each object is assigned:
 - Class label (e.g., Fastener Absence)
 - Unique instance mask
- Enables multi-class and multi-instance learning.

Mask Creation Process

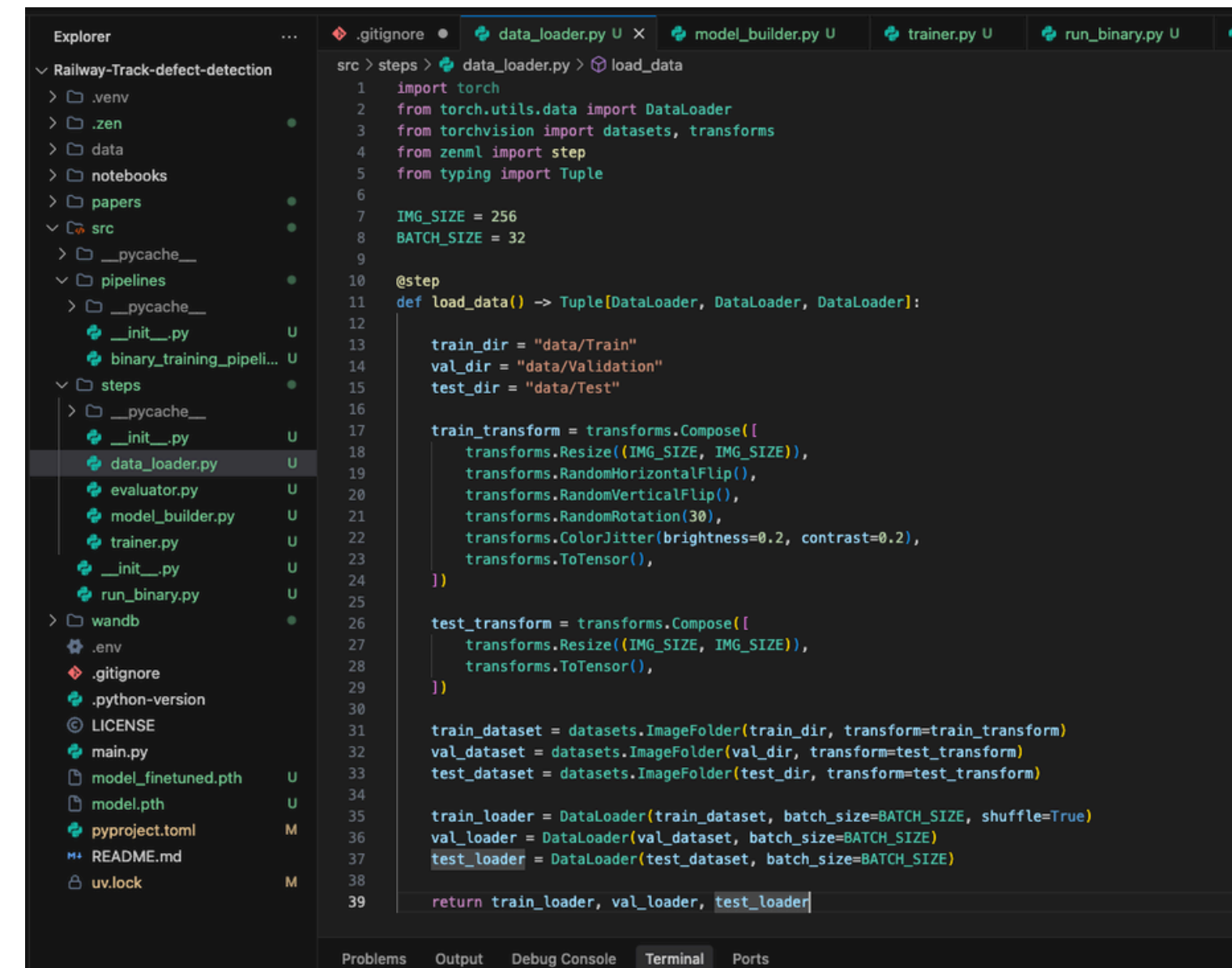
- Manual polygon drawing around defect region.
- Zoom-based fine boundary refinement for accuracy.
- Background automatically excluded from mask.
- Quality check performed before saving annotations



Data Pipeline Architecture & Workflow

Orchestration

- Implemented a PyTorch-based data loading pipeline using custom Dataset and DataLoader classes for efficient batch processing and GPU training.
- Integrated Albumentations for advanced data augmentation, including geometric transformations, brightness/contrast adjustments, and normalization to improve model generalization.
- Applied augmentation consistently to both images and corresponding segmentation masks to preserve pixel-level alignment.
- Structured the pipeline to support multi-class and instance-level segmentation data, ensuring compatibility with YOLO-based segmentation formats.
- Orchestrated the complete data preprocessing and training workflow using ZenML, enabling reproducible, modular, and scalable ML pipeline management.



```
src > steps > data_loader.py > load_data
1  import torch
2  from torch.utils.data import DataLoader
3  from torchvision import datasets, transforms
4  from zenml import step
5  from typing import Tuple
6
7  IMG_SIZE = 256
8  BATCH_SIZE = 32
9
10 @step
11 def load_data() -> Tuple[DataLoader, DataLoader, DataLoader]:
12
13     train_dir = "data/Train"
14     val_dir = "data/Validation"
15     test_dir = "data/Test"
16
17     train_transform = transforms.Compose([
18         transforms.Resize(IMG_SIZE, IMG_SIZE),
19         transforms.RandomHorizontalFlip(),
20         transforms.RandomVerticalFlip(),
21         transforms.RandomRotation(30),
22         transforms.ColorJitter(brightness=0.2, contrast=0.2),
23         transforms.ToTensor(),
24     ])
25
26     test_transform = transforms.Compose([
27         transforms.Resize(IMG_SIZE, IMG_SIZE),
28         transforms.ToTensor(),
29     ])
30
31     train_dataset = datasets.ImageFolder(train_dir, transform=train_transform)
32     val_dataset = datasets.ImageFolder(val_dir, transform=test_transform)
33     test_dataset = datasets.ImageFolder(test_dir, transform=test_transform)
34
35     train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
36     val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE)
37     test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE)
38
39     return train_loader, val_loader, test_loader
```

Codebase

```
src > steps > trainer.py > ...
5 import numpy as np
6 from sklearn.metrics import confusion_matrix, precision_score, recall_score, roc_auc_score
7 import matplotlib.pyplot as plt
8 from zenml import step
9
10 # --- Device ---
11 if torch.cuda.is_available():
12     DEVICE = "cuda"
13 elif torch.backends.mps.is_available():
14     DEVICE = "mps"
15 else:
16     DEVICE = "cpu"
17
18 # --- Hyperparameters ---
19 EPOCHS_STAGE1 = 5
20 EPOCHS_STAGE2 = 10
21 LR_STAGE1 = 1e-3
22 LR_STAGE2 = 1e-5
23
24 @step
25 def train_model(model: torch.nn.Module,
26                 train_loader,
27                 val_loader) -> torch.nn.Module:
28     wandb.init(
29         project="track-defect-detection",
30         config={
31             "epochs_stage1": EPOCHS_STAGE1,
32             "epochs_stage2": EPOCHS_STAGE2,
33             "lr_stage1": LR_STAGE1,
34             "lr_stage2": LR_STAGE2,
35             "architecture": "MobileNetV2"
36         },
37         reinit=True
38     )
39     model = model.to(DEVICE)
40
41     # STAGE 1 - Freeze Backbone
42     # STAGE 2 - Train Head
43     for param in model.features.parameters():
44         param.requires_grad = False
45
46     optimizer = optim.Adam(model.parameters(), lr=LR_STAGE1)
```

```
Explorer
Railway-Track-defect-detection
  .venv
  .zen
  data
  notebooks
  papers
  src
  __pycache__
  pipelines
  __pycache__
  _init_.py
  binary_training_pipeline.py
  steps
  __pycache__
  _init_.py
  data_loader.py
  evaluator.py
  model_builder.py
  trainer.py
  _init_.py
  run_binary.py
  wandb
  .env
  .gitignore
  .python-version
  LICENSE
  main.py
  model_finetuned.pth
  model.pth
  pyproject.toml
  README.md
  uv.lock

  .gitignore
  1 # Byte-compiled / optimized / DLL files
  2 __pycache__
  3 *.py[cod]
  4 $*.class
  5
  6 # C extensions
  7 *.so
  8
  9 # Distribution / packaging
  10 .Python
  11 build
  12 develop-eggs/
  13 dist/
  14 downloads/
  15 eggs/
  16 .eggs/
  17 lib/
  18 lib64/
  19 parts/
  20 sdist/
  21 var/
  22 wheels/
  23 share/python-wheels/
  24 *.egg-info/
  25 .installed.cfg
  26 *.egg
  27 MANIFEST
  28
  29 #assets
  30 data/
  31
  32 # PyInstaller
  33 # Usually these files are written by a python script from
  34 # before PyInstaller builds the exe, so as to inject data
  35 # before PyInstaller builds the exe, so as to inject data
  36 *.manifest
  37 *.spec
  38
  39 # Installer logs
  40 pip-log.txt
  41 pip-delete-this-directory.txt
  42
  43 # Unit test / coverage reports
  44 htmlcov/
  45 .tox/
  46 .nox/
  47 .coverage
```

```
src > steps > evaluator.py > evaluate_model > total
1 import torch
2 from zenml import step
3
4 DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
5
6 @step
7 def evaluate_model(model: torch.nn.Module, test_loader) -> float:
8
9     model.eval()
10    model.to(DEVICE)
11
12    correct = 0
13    total = 0
14
15    with torch.no_grad():
16        for images, labels in test_loader:
17            images = images.to(DEVICE)
18            labels = labels.to(DEVICE)
19
20            outputs = model(images)
21            preds = torch.sigmoid(outputs) > 0.5
22            correct += (preds.squeeze() == labels).sum().item()
23            total += labels.size(0)
24
25    accuracy = correct / total
26    print(f"Test Accuracy: {accuracy:.4f}")
27
28    return accuracy
```

Objectives

- Develop a panoptic segmentation model for railway track defect detection.
- Move beyond binary classification to enable pixel-level, multi-class defect identification.
- Curate and annotate a custom segmentation dataset to support instance-level learning.
- Build a scalable and reproducible ML pipeline using PyTorch, Albumentations, and ZenML.
- Design a system capable of detecting multiple defects within a single image.

Objectives

Primary objectives

- Develop a vision-based railway track defect detection system using Mask2Former for accurate surface anomaly segmentation.
- Apply panoptic segmentation to simultaneously classify defect types and uniquely identify individual defect instances.
- Evaluate performance against CNN-based methods to demonstrate improvements in localization and interpretability.

Secondary objectives

- Develop a lightweight web-based user interface to visualize inputs, run model inference, and display defect segmentation results.
- Optimize the trained model using quantization or model compression techniques to reduce computational and memory requirements.
- Explore deployment feasibility on resource-constrained environments to support real-time or near-real-time inspection use cases.

Expected Outcomes

- A trained panoptic segmentation model capable of identifying and localizing railway defects.
- Accurate pixel-level masks for:
 - Cracks
 - Fastener absence
 - Other structural anomalies
- A reusable and extendable segmentation dataset for future research and improvements.
- A modular ML pipeline ready for:
 - Model experimentation
 - Performance evaluation
 - Future deployment
- Foundation for real-world applications in automated railway inspection systems.

Expected Outcome

1. Accurate Defect Detection

- Identify different types of rail defects such as cracks, rust, and fastening issues.
- Detect each defect as a separate instance for precise tracking.

2. Predictive Maintenance Capability

- Detect defects to plan maintenance before failures occur.
- Reduce costly emergency repairs and operational downtime.

3. Improved Panoptic Segmentation Performance

- Achieve higher Panoptic Quality (PQ) compared to traditional CNN-based models.
- Maintain fast inference speed suitable for real-time or near-real-time inspections.

Next steps

- Model Training
- Train the panoptic segmentation model using the curated dataset with optimized hyperparameters and appropriate loss functions (e.g., segmentation + classification losses).
- Model Fine-Tuning
- Fine-tune pretrained backbones (e.g., ResNet / Transformer-based models) to improve feature extraction and accelerate convergence.
- Hyperparameter Optimization
- Experiment with learning rates, batch sizes, augmentation strategies, and optimizer configurations to maximize performance.
- Model Evaluation
- Evaluate performance using segmentation-specific metrics such as:
 - mIoU (Mean Intersection over Union)
 - Dice Coefficient
 - Precision, Recall, F1-Score
- Validation & Generalization Testing
- Test the model on unseen images and multi-defect scenarios to assess robustness and real-world applicability.

Thank you